

Smart Calculator

A calculator for the command line, written in Python

VITyarthi Build Your Own Project

Name: Shivansh Sharma

Reg. No.:26MIM10224

Why I made this

I wanted one calculator that can do more than $2 + 2$.

- Basic calculators can't take a whole expression or remember anything
- Unit conversion, equations and statistics usually mean three different apps
- Many quick Python calculators use `eval()`, which runs anything you type into it

So I built one tool that does all of it, and does it safely.

Goals and who it is for

What I wanted

- Work out full expressions without using eval()
- Remember variables and past answers
- Convert units, solve equations, do statistics
- Show clear errors instead of crashing
- Use only the standard library, and test everything

Who would use it

- Students doing maths, science and engineering homework
- Programmers and analysts who live in the terminal
- Anyone who wants a safe calculator with a history

What it can do

Expressions: brackets, functions, variables, the last answer

Unit converter: 8 kinds, like km to miles

Equation solver: linear, quadratic and systems of equations

Statistics: mean, median, mode, standard deviation

History: view, search and delete, saved between sessions

Logging: every calculation goes in a log file

Rules I set

- No eval(), so typed code can't run
- Never crash on bad input
- Clear error messages
- Standard library only

Requirements

What it must do

- FR1 Work out expressions with functions and constants
- FR2 Store variables and the last answer, deg/rad mode
- FR3 Convert units in 8 categories
- FR4 Solve equations and linear systems
- FR5 Give descriptive statistics
- FR6 Keep a history: add, view, search, delete, save
- FR7 Work interactively and as a one-line command

How well it must do it

Security: no eval or exec, input limits

Reliability: never crashes, safe file writes

Usability: clear errors, :help, 2x works

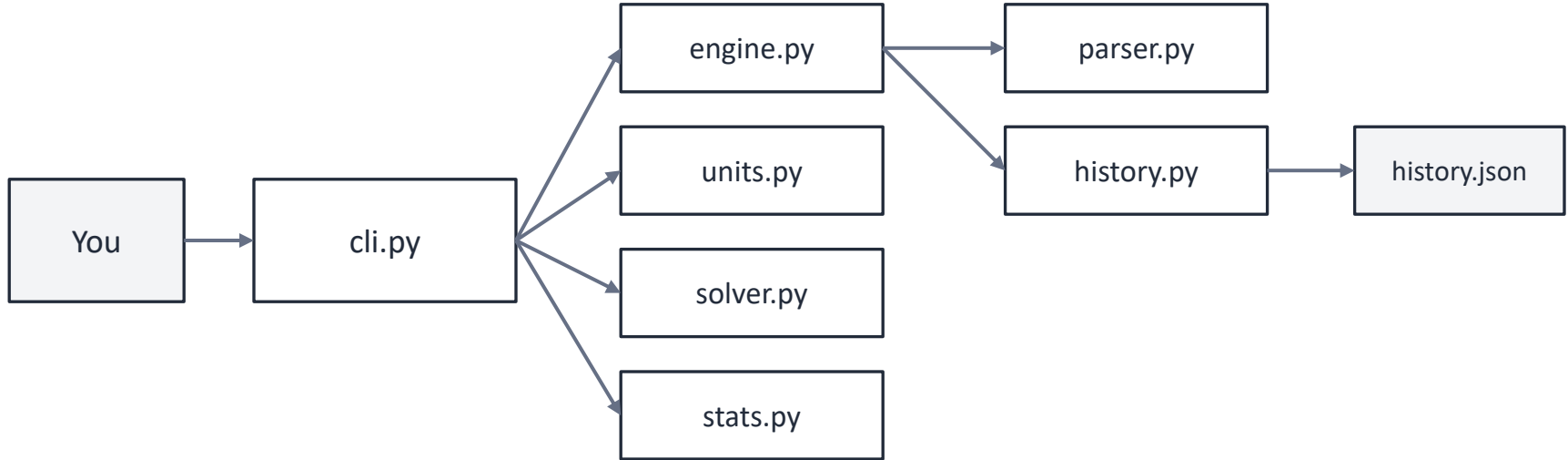
Performance: under 10 ms per expression

Maintainability: 11 small modules, tests, CI

Logging: every calculation and failure

Efficiency: stdlib only, history capped at 500

How it is put together



cli.py takes what you type and hands it to the right module. The engine uses the parser to work out the answer and history to save it. The solver uses the parser too.

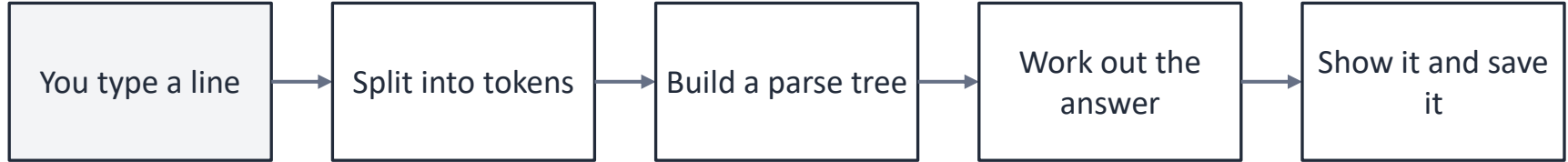
Project structure and tools

```
smart-calculator/  
  main.py  
  smart_calculator/  
    cli.py  engine.py  parser.py  
    functions.py  units.py  solver.py  
    stats.py  history.py  formatting.py  
    exceptions.py  logger.py  
  tests/          7 test files  
  docs/           design + report guide  
  README.md  statement.md  
  .github/workflows/tests.yml
```

Tools used

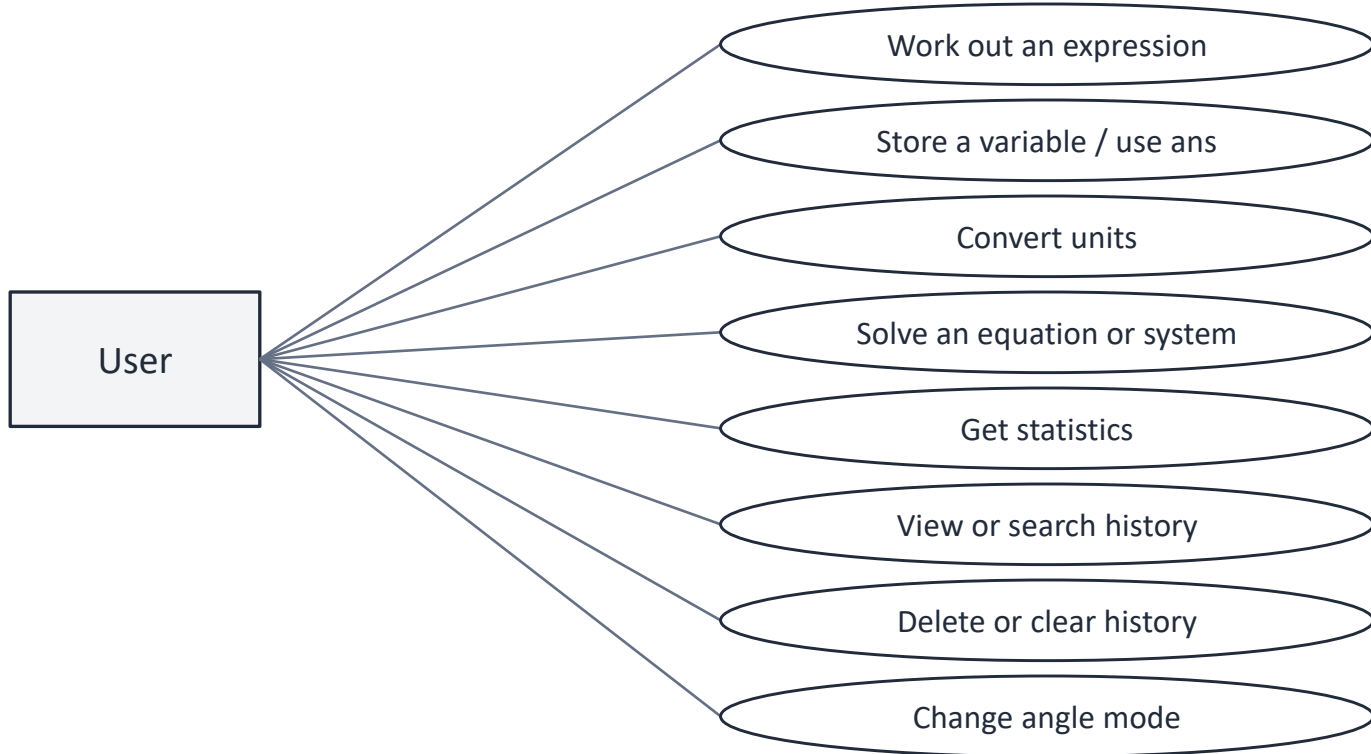
- Python 3.9 or newer
- Standard library only, no pip install
- unittest for the tests
- Git and GitHub
- GitHub Actions to run the tests

What happens when you type



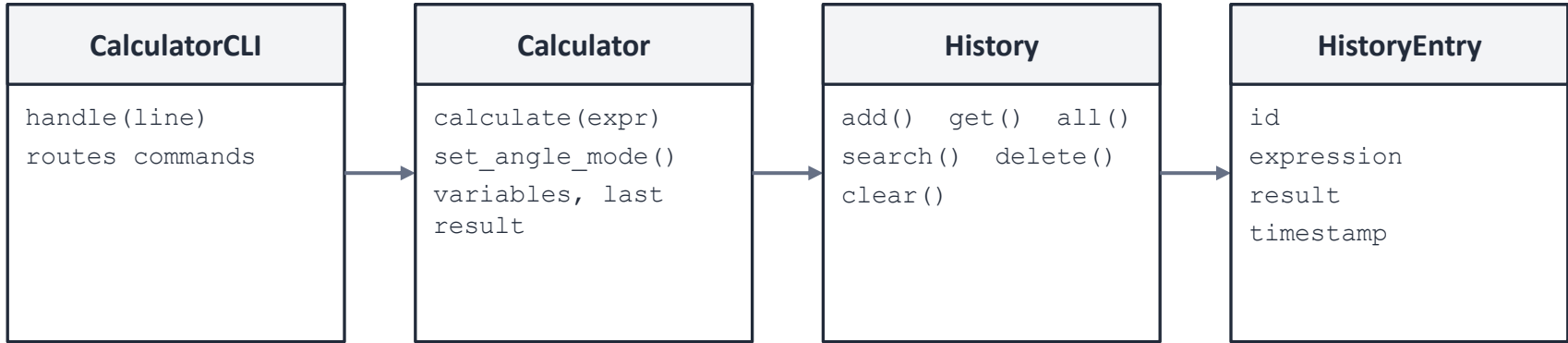
- Lines that start with a colon, like `:convert` or `:solve`, go straight to that module
- If anything goes wrong you get a short Error message and can keep going

What a user can do



One person, eight things they can ask for. Everything goes through the same text prompt.

The main classes



Errors have a family too

`CalculatorError` is the parent. `ParseError`, `EvaluationError`, `UnitError`, `SolverError`, `StatsError` and `HistoryError` all come from it, so the CLI can catch one type and show a clean message.

Choices I made, and why

A parser instead of `eval()`. It is safe by design, gives error positions, and I got to learn how tokenizers and grammars work.

The solver reuses the evaluator. It tries $f(x) = \text{left} - \text{right}$ at three points to find a, b and c, then checks with three more. No second parser needed.

Gaussian elimination with pivoting. Stable numbers, and it notices when a system has no single answer.

History in a JSON file. Easy to read, no database, and written safely so a crash cannot corrupt it.

The CLI returns strings. That made the whole interface easy to unit test.

Keeping it safe and stable

Limits

- Expressions up to 500 characters
- Exponents up to 10,000
- Factorials up to 1,000
- History keeps the last 500 entries

When something goes wrong

- Each module raises its own kind of error
- The prompt catches it and prints a short message
- A broken history file is ignored, not fatal
- A rotating log records every calculation and failure

Trying it out

```
calc> 2+3*4
14
calc> :convert 10 km to mi
10 km = 6.21371192237 mi
calc> :solve x^2 - 5x + 6 = 0
x = 3
x = 2
calc> :stats 4 8 15 16 23 42
mean 18    median 15.5    ...
calc> 1/0
Error: Division by zero
calc> __import__('os')
Error: Unexpected character "'"
```

This is real output from the program. The last line tries to run code, and it gets rejected.

To run it:

```
python main.py
```

Commands at a glance

Type this	To do this
<code>2+3*4 sqrt(16) 5!</code>	Work out an expression
<code>x = 5 2x+1 ans*2</code>	Variables and the last answer
<code>:convert 10 km to mi</code>	Convert units (:units lists them)
<code>:solve 2x+3=7</code>	Linear or quadratic equation
<code>:system 2,1,5; 1,-1,1</code>	Linear system
<code>:stats 4 8 15 16 23 42</code>	Descriptive statistics
<code>:mode deg</code>	Switch between deg and rad
<code>:history 5 :search x :clear</code>	View, search, delete, clear history

Testing

38 tests, all passing

GitHub Actions runs them on every push, on Python 3.9, 3.10, 3.11 and 3.12. One test tries to sneak code in, to make sure it gets blocked.

```
python -m unittest discover -s tests -v
```

test_parser

order of operations, brackets, security

test_engine

variables, ans, angle modes

test_units

conversions, unknown units

test_solver

roots, complex roots, bad systems

test_stats

every statistic, empty input

test_history

save, search, delete, broken file

test_cli

commands and error output

What was hard, what I learned

What was hard

- Making $2x$ work like $2 * x$
- Solving equations without an algebra library
- Not crashing when the history file is broken

What I learned

- How a tokenizer and parser turn text into something a program can work out
- Why `eval()` is risky, and how to avoid it
- Small files with one job are easier to test
- Git and automatic tests catch mistakes early

If I had more time

- A graphical or web version
- Graphing of functions
- Symbolic algebra, like simplifying and calculus
- Equations above degree 2

Thank you!

<https://github.com/shivanshsharma894/SMART-CALCULATOR>